4190.407 Algorithms HW 4-2

Yeonjun Kim

2024-13755

Dept. of Computer Science and Engineering

2025-11-05

In this assignment, I implemented the shortest path algorithms; Dijkstra and Bellman-Ford.

Before running the program, you should place `input.txt` in this directory. In `input.txt`, you write the graph edges and the starting node. I verified the implementation for edge cases in `test/` You can run the program by `$ python3 202413755_Task4.py`, and see the results in `output.txt`.

Theoretical Comparison

The two have different theoretical complexities, as shown in **Table 1**. V denotes the number of vertices, and E denotes the number of edges in the graph.

Table 1 time complexity comparison in big-O notation.

	Dijkstra		**Bellman-Ford**
	Using array	**Using priority queue (heap)**	
Time Complexity	$O(V^2)$	$O(E \log V)$	$O(VE)$

In general, Dijkstra algorithm outperforms Bellman-Ford when effectively implemented with priority queue. However, Bellman-Ford has advantage over Dijkstra as it can be used when there are negative weight edges and it can detect negative weight cycles. Meanwhile, Dijkstra algorithm fails when there are negative weight cycles.

Experimental Setup

The Dijkstra algorithm was implemented with priority queue (`pqueue` in Python), and Bellman-Ford was implemented in the way in lecture slides.

In my timing experiments, I generated graphs with different number of vertices. from V=100 to V=1000. Then I randomly generated edges between vertices. The density of graph was fixed to E=5V.

Performance Comparison

The comparison result is shown in **Figure 1**. Dijkstra algorithm generally performed faster than Bellman-Ford algorithm, and the overhead grew more gradually than Bellman-Ford, as expected in theoretical comparison.

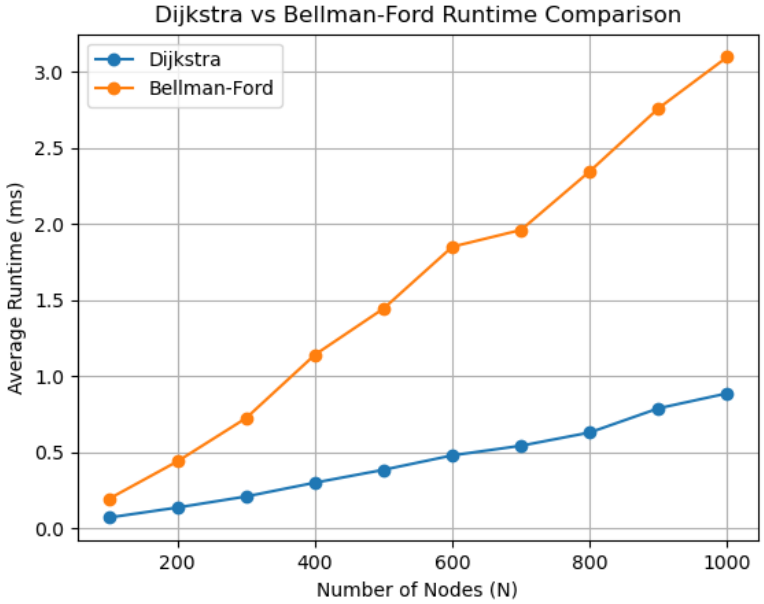

Figure 1 runtime comparison results.

Analysis & Conclusion

In this experiment, I could understand key ideas and characteristics of the two shortest path algorithms. I could see Dijkstra algorithm was significantly faster than Bellman-Ford, though it may fail on graphs with negative weights.

Also, I learned that the performance of Dijkstra largely depended on the ways to find and remove the minimum element. I observed that some algorithms are used as building blocks of other advanced algorithms, enhancing the overall performance.